

SGT UNIVERSITY
School of Engineering and Technology
Department of CSE

DATABASE MANAGEMENT SYSTEMS LABORATORY

Experiment 5

Aggregate Functions

Field	Details
Student	Harshit Khemani
Roll No.	241302081
Programme	B.Tech CSE (AI/ML)
Section	Section - C
Lab Date	30 September 2026
Dataset	employee (16 rows; CSE, Mechanical, ECE, Civil)
Methods	SUM, AVG, MAX, MIN, COUNT, GROUP BY, HAVING

1. Aim

To create an employee table, insert 16 rows across four departments, and answer the lab questions with aggregate functions: COUNT, SUM, AVG, MAX and MIN, including GROUP BY and HAVING. Nested queries are Experiment 6.

2. Theory

2.1 Aggregate functions

An aggregate function computes one value from many rows. COUNT(*) counts rows. SUM(salary) adds the salary column. AVG(salary) is the arithmetic mean. MAX and MIN return the largest and smallest values. Without GROUP BY, each aggregate covers the whole table and the query returns one row. SQLite prints this table's average as 56125.0 because AVG returns a real number even when the mean is a whole number of rupees. SUM of these whole-number salaries prints as the integer 898000.

2.2 COUNT and COUNT DISTINCT

COUNT(*) counts rows, so question 3 returns 16. COUNT(DISTINCT department) counts different values of that column. Ten of the sixteen rows repeat a department name, so question 4 returns 4: CSE, Mechanical, ECE and Civil.

2.3 GROUP BY

GROUP BY department makes one group per distinct department, then evaluates the aggregates inside each group. Every column in the SELECT list must be either grouped or aggregated. Question 8 therefore returns four rows, each with its own count, total, average, highest salary and lowest salary.

2.4 WHERE and HAVING

WHERE filters rows before the aggregates run. Question 11 uses it to keep only CSE, then computes one count, one total and one average for those five rows. HAVING filters groups after the aggregates are computed, so it can test AVG(salary) or COUNT(*). WHERE AVG(salary) > 55000 is illegal. Question 9 keeps departments whose average is above 55000 (Civil at 45000.0 drops out). Question 10 keeps departments with more than three employees (Civil, with three, drops out). Question 12 groups by city and keeps the five cities that appear twice.

3. Schema

One table, employee, holds six attributes. sr_no is the employee number (primary key). department and city are stored as names so GROUP BY results can be read without a second lookup. salary is a positive monthly amount in rupees.

Attribute	Type	Constraint	Role
sr_no	INTEGER	PRIMARY KEY	Employee number
employee_name	TEXT	NOT NULL	Employee name
job	TEXT	NOT NULL	Job title
department	TEXT	NOT NULL	Department name
salary	NUMERIC(10,2)	NOT NULL, CHECK > 0	Monthly salary (Rs)
city	TEXT	NOT NULL	Work city

4. Questions

Twelve lab questions. Questions 1 and 2 build the table. Questions 3–7 are aggregates over every employee. Question 8 groups by department. Questions 9, 10 and 12 filter groups with HAVING. Question 11 filters rows with WHERE before the aggregates, which is how a single department is summarised.

1. Create a table employee with the following attributes:
 - sr no
 - employee name
 - job
 - department
 - salary
 - city
2. Insert 16 rows in the above mentioned table.
Four departments: CSE (5), Mechanical (4), ECE (4) and Civil (3). Salaries are spread so each aggregate and each HAVING filter returns a different set. Five cities have two employees; the other cities have one.
3. Display the total number of employees from the employee table using COUNT.
4. Display the number of distinct departments from the employee table using COUNT.
5. Display the total salary of all employees from the employee table using SUM.
6. Display the average salary of all employees from the employee table using AVG.
7. Display the maximum and minimum salary from the employee table using MAX and MIN.
8. Display the number of employees, total salary, average salary, highest salary and lowest salary of each department using COUNT, SUM, AVG, MAX, MIN and GROUP BY.
9. Display the department and average salary of departments whose average salary is greater than 55000 using GROUP BY and HAVING.
10. Display the department and the number of employees for departments that have more than 3 employees using GROUP BY and HAVING.
11. Display the number of employees, total salary and average salary of the CSE department using WHERE with COUNT, SUM and AVG.
12. Display the cities that have more than one employee using GROUP BY and HAVING.

5. Procedure

Drop employee if it already exists so the script can be re-run. Create the table, insert the 16 rows, then run each query in order. Grouped results are ordered so the same script prints the same rows on every run. The SQL for each question:

Q. No.	SQL solution
1	<pre>CREATE TABLE employee (sr_no INTEGER PRIMARY KEY, employee_name TEXT NOT NULL, job TEXT NOT NULL, department TEXT NOT NULL, salary NUMERIC(10, 2) NOT NULL CHECK (salary > 0), city TEXT NOT NULL);</pre>

Q. No.	SQL solution
2	<pre> INSERT INTO employee (sr_no, employee_name, job, department, salary, city) VALUES (1, 'Rajesh Kumar', 'Professor', 'CSE', 92000.00, 'Delhi'), (2, 'Ananya Sharma', 'Associate Professor', 'CSE', 78000.00, 'Bangalore'), (3, 'Fatima Khan', 'Assistant Professor', 'CSE', 56000.00, 'Bangalore'), (4, 'Karan Iyer', 'Lab Instructor', 'CSE', 42000.00, 'Mumbai'), (5, 'Dev Patel', 'Teaching Assistant', 'CSE', 28000.00, 'Ahmedabad'), (6, 'Vikram Singh', 'Professor', 'Mechanical', 88000.00, 'Hyderabad'), (7, 'Rohan Mehta', 'Associate Professor', 'Mechanical', 64000.00, 'Mumbai'), (8, 'Arjun Patel', 'Assistant Professor', 'Mechanical', 48000.00, 'Pune'), (9, 'Mohit Jain', 'Lab Instructor', 'Mechanical', 32000.00, 'Jaipur'), (10, 'Priya Nair', 'Professor', 'ECE', 90000.00, 'Chennai'), (11, 'Neha Gupta', 'Associate Professor', 'ECE', 58000.00, 'Delhi'), (12, 'Meera Joshi', 'Assistant Professor', 'ECE', 51000.00, 'Chennai'), (13, 'Kavya Menon', 'Lab Instructor', 'ECE', 36000.00, 'Kochi'), (14, 'Sneha Reddy', 'Associate Professor', 'Civil', 60000.00, 'Hyderabad'), (15, 'Rahul Das', 'Assistant Professor', 'Civil', 45000.00, 'Kolkata'), (16, 'Tushar Rao', 'Lab Instructor', 'Civil', 30000.00, 'Nagpur'); </pre>
3	<pre> SELECT COUNT(*) AS employee_count FROM employee; </pre>
4	<pre> SELECT COUNT(DISTINCT department) AS department_count FROM employee; </pre>
5	<pre> SELECT SUM(salary) AS total_salary FROM employee; </pre>
6	<pre> SELECT AVG(salary) AS average_salary FROM employee; </pre>
7	<pre> SELECT MAX(salary) AS maximum_salary, MIN(salary) AS minimum_salary FROM employee; </pre>
8	<pre> SELECT department, COUNT(*) AS employee_count, SUM(salary) AS total_salary, AVG(salary) AS average_salary, MAX(salary) AS highest_salary, MIN(salary) AS lowest_salary FROM employee GROUP BY department ORDER BY department; </pre>
9	<pre> SELECT department, AVG(salary) AS average_salary FROM employee GROUP BY department HAVING AVG(salary) > 55000 ORDER BY department; </pre>
10	<pre> SELECT department, COUNT(*) AS employee_count FROM employee GROUP BY department HAVING COUNT(*) > 3 ORDER BY department; </pre>
11	<pre> SELECT COUNT(*) AS employee_count, SUM(salary) AS total_salary, AVG(salary) AS average_salary FROM employee WHERE department = 'CSE'; </pre>
12	<pre> SELECT city, COUNT(*) AS employee_count FROM employee GROUP BY city HAVING COUNT(*) > 1 ORDER BY city; </pre>

6. Source Code

SQLite script used in the lab compiler (30-09-2026/employee.sql). For the class SQL Server, run employee.sqlserver.sql.

```
-- =====
-- DBMS Lab · 30-09-2026 · Experiment 5
```

```
-- Employee table: CREATE, seed 16 rows, then aggregate functions
-- SUM, AVG, MAX, MIN, COUNT, GROUP BY, HAVING
-- SQLite (sql.js compiler / DB Browser / sqlite3)
-- For the class SQL Server, run employee.sqlserver.sql instead.
-- Nested queries are Experiment 6 (30-09-2026-2).
--
-- Lab questions
-- 1. Create a table employee with the following attributes:
--   sr no, employee name, job, department, salary, city
-- 2. Insert 16 rows in the above mentioned table
-- 3. Display the total number of employees using COUNT
-- 4. Display the number of distinct departments using COUNT
-- 5. Display the total salary of all employees using SUM
-- 6. Display the average salary of all employees using AVG
-- 7. Display the maximum and minimum salary using MAX and MIN
-- 8. For each department, display COUNT, SUM, AVG, MAX and MIN
--   using GROUP BY
-- 9. Display departments whose average salary is greater than
--   55000 using GROUP BY and HAVING
-- 10. Display departments that have more than 3 employees
--   using GROUP BY and HAVING
-- 11. Display the number of employees, total salary and average
--   salary of the CSE department
-- 12. Display cities that have more than one employee using
--   GROUP BY and HAVING
-- =====
DROP TABLE IF EXISTS employee;

-- -----
-- 1. Create a table employee with the following attributes:
--   sr no, employee name, job, department, salary, city
-- -----
CREATE TABLE employee (
    sr_no      INTEGER PRIMARY KEY,
    employee_name TEXT NOT NULL,
    job        TEXT NOT NULL,
    department  TEXT NOT NULL,
    salary      NUMERIC(10, 2) NOT NULL CHECK (salary > 0),
    city       TEXT NOT NULL
);

-- -----
-- 2. Insert 16 rows in the above mentioned table
--   • four departments: CSE (5), Mechanical (4), ECE (4), Civil (3)
--   • salaries vary so SUM, AVG, MAX, MIN and HAVING differ
--   • five cities have two employees; the rest have one
-- -----
INSERT INTO employee
(sr_no, employee_name, job, department, salary, city)
VALUES
(1, 'Rajesh Kumar', 'Professor', 'CSE', 92000.00, 'Delhi'),
(2, 'Ananya Sharma', 'Associate Professor', 'CSE', 78000.00, 'Bangalore'),
(3, 'Fatima Khan', 'Assistant Professor', 'CSE', 56000.00, 'Bangalore'),
(4, 'Karan Iyer', 'Lab Instructor', 'CSE', 42000.00, 'Mumbai'),
(5, 'Dev Patel', 'Teaching Assistant', 'CSE', 28000.00, 'Ahmedabad'),
(6, 'Vikram Singh', 'Professor', 'Mechanical', 88000.00, 'Hyderabad'),
(7, 'Rohan Mehta', 'Associate Professor', 'Mechanical', 64000.00, 'Mumbai'),
(8, 'Arjun Patel', 'Assistant Professor', 'Mechanical', 48000.00, 'Pune'),
(9, 'Mohit Jain', 'Lab Instructor', 'Mechanical', 32000.00, 'Jaipur'),
(10, 'Priya Nair', 'Professor', 'ECE', 90000.00, 'Chennai'),
(11, 'Neha Gupta', 'Associate Professor', 'ECE', 58000.00, 'Delhi'),
(12, 'Meera Joshi', 'Assistant Professor', 'ECE', 51000.00, 'Chennai'),
(13, 'Kavya Menon', 'Lab Instructor', 'ECE', 36000.00, 'Kochi'),
(14, 'Sneha Reddy', 'Associate Professor', 'Civil', 60000.00, 'Hyderabad'),
(15, 'Rahul Das', 'Assistant Professor', 'Civil', 45000.00, 'Kolkata'),
(16, 'Tushar Rao', 'Lab Instructor', 'Civil', 30000.00, 'Nagpur');

-- -----
-- 3. Display the total number of employees from the employee
--   table using COUNT
-- -----
-- Total number of employees
SELECT COUNT(*) AS employee_count
FROM employee;

-- -----
-- 4. Display the number of distinct departments using COUNT
-- -----
-- Number of distinct departments
SELECT COUNT(DISTINCT department) AS department_count
FROM employee;

-- -----
-- 5. Display the total salary of all employees from the
--   employee table using SUM
-- -----
-- Total salary of all employees
SELECT SUM(salary) AS total_salary
FROM employee;
-- -----
```

```

-- 6. Display the average salary of all employees from the
-- employee table using AVG
-----
-- Average salary of all employees
SELECT AVG(salary) AS average_salary
FROM employee;

-----
-- 7. Display the maximum and minimum salary from the employee
-- table using MAX and MIN
-----
-- Maximum and minimum salary
SELECT MAX(salary) AS maximum_salary,
       MIN(salary) AS minimum_salary
FROM employee;

-----
-- 8. Display the number of employees, total salary, average
-- salary, highest salary and lowest salary of each
-- department using COUNT, SUM, AVG, MAX, MIN and GROUP BY
-----
-- Aggregates for each department
SELECT department,
       COUNT(*) AS employee_count,
       SUM(salary) AS total_salary,
       AVG(salary) AS average_salary,
       MAX(salary) AS highest_salary,
       MIN(salary) AS lowest_salary
FROM employee
GROUP BY department
ORDER BY department;

-----
-- 9. Display the department and average salary of departments
-- whose average salary is greater than 55000 using
-- GROUP BY and HAVING
-----
-- Departments with average salary above 55000
SELECT department,
       AVG(salary) AS average_salary
FROM employee
GROUP BY department
HAVING AVG(salary) > 55000
ORDER BY department;

-----
-- 10. Display the department and the number of employees for
-- departments that have more than 3 employees using
-- GROUP BY and HAVING
-----
-- Departments with more than 3 employees
SELECT department,
       COUNT(*) AS employee_count
FROM employee
GROUP BY department
HAVING COUNT(*) > 3
ORDER BY department;

-----
-- 11. Display the number of employees, total salary and
-- average salary of the CSE department
-----
-- CSE department aggregates
SELECT COUNT(*) AS employee_count,
       SUM(salary) AS total_salary,
       AVG(salary) AS average_salary
FROM employee
WHERE department = 'CSE';

-----
-- 12. Display cities that have more than one employee using
-- GROUP BY and HAVING
-----
-- Cities with more than one employee
SELECT city,
       COUNT(*) AS employee_count
FROM employee
GROUP BY city
HAVING COUNT(*) > 1
ORDER BY city;

```

7. Output

The script was executed in SQLite in memory. CREATE prints a status line and INSERT prints the row count. Each SELECT prints the result set that SQLite returned. Whole-number salaries print as integers; AVG prints a real (56125.0, 59200.0, 58750.0, and so on).

```

SQL> DROP TABLE IF EXISTS employee;
-- OK

```

```

SQL> CREATE TABLE employee (
    sr_no          INTEGER PRIMARY KEY,
    employee_name  TEXT NOT NULL,
    job            TEXT NOT NULL,
    department     TEXT NOT NULL,
    salary         NUMERIC(10, 2) NOT NULL CHECK (salary > 0),
    city           TEXT NOT NULL
);
-- OK

SQL> INSERT INTO employee
(sr_no, employee_name, job, department, salary, city)
VALUES
(1, 'Rajesh Kumar', 'Professor', 'CSE', 92000.00, 'Delhi'),
(2, 'Ananya Sharma', 'Associate Professor', 'CSE', 78000.00, 'Bangalore'),
(3, 'Fatima Khan', 'Assistant Professor', 'CSE', 56000.00, 'Bangalore'),
(4, 'Karan Iyer', 'Lab Instructor', 'CSE', 42000.00, 'Mumbai'),
(5, 'Dev Patel', 'Teaching Assistant', 'CSE', 28000.00, 'Ahmedabad'),
(6, 'Vikram Singh', 'Professor', 'Mechanical', 88000.00, 'Hyderabad'),
(7, 'Rohan Mehta', 'Associate Professor', 'Mechanical', 64000.00, 'Mumbai'),
(8, 'Arjun Patel', 'Assistant Professor', 'Mechanical', 48000.00, 'Pune'),
(9, 'Mohit Jain', 'Lab Instructor', 'Mechanical', 32000.00, 'Jaipur'),
(10, 'Priya Nair', 'Professor', 'ECE', 90000.00, 'Chennai'),
(11, 'Neha Gupta', 'Associate Professor', 'ECE', 58000.00, 'Delhi'),
(12, 'Meera Joshi', 'Assistant Professor', 'ECE', 51000.00, 'Chennai'),
(13, 'Kavya Menon', 'Lab Instructor', 'ECE', 36000.00, 'Kochi'),
(14, 'Sneha Reddy', 'Associate Professor', 'Civil', 60000.00, 'Hyderabad'),
(15, 'Rahul Das', 'Assistant Professor', 'Civil', 45000.00, 'Kolkata'),
(16, 'Tushar Rao', 'Lab Instructor', 'Civil', 30000.00, 'Nagpur');
-- 16 row(s) inserted

SQL> SELECT COUNT(*) AS employee_count
FROM employee;
employee_count
-----
16

SQL> SELECT COUNT(DISTINCT department) AS department_count
FROM employee;
department_count
-----
4

SQL> SELECT SUM(salary) AS total_salary
FROM employee;
total_salary
-----
898000

SQL> SELECT AVG(salary) AS average_salary
FROM employee;
average_salary
-----
56125.0

SQL> SELECT MAX(salary) AS maximum_salary,
MIN(salary) AS minimum_salary
FROM employee;
maximum_salary  minimum_salary
-----
92000           28000

SQL> SELECT department,
COUNT(*) AS employee_count,
SUM(salary) AS total_salary,
AVG(salary) AS average_salary,
MAX(salary) AS highest_salary,
MIN(salary) AS lowest_salary
FROM employee
GROUP BY department
ORDER BY department;
department  employee_count  total_salary  average_salary  highest_salary  lowest_salary
-----
CSE         5              296000       59200.0        92000          28000
Civil       3              135000       45000.0        60000          30000
ECE         4              235000       58750.0        90000          36000
Mechanical  4              232000       58000.0        88000          32000

SQL> SELECT department,
AVG(salary) AS average_salary
FROM employee
GROUP BY department
HAVING AVG(salary) > 55000
ORDER BY department;
department  average_salary
-----
CSE         59200.0
ECE         58750.0
Mechanical  58000.0

SQL> SELECT department,
COUNT(*) AS employee_count
FROM employee
GROUP BY department
HAVING COUNT(*) > 3
ORDER BY department;
department  employee_count
-----
CSE         5
ECE         4
Mechanical  4

SQL> SELECT COUNT(*) AS employee_count,
SUM(salary) AS total_salary,
AVG(salary) AS average_salary

```

```

FROM employee
WHERE department = 'CSE';
employee_count  total_salary  average_salary
-----
5                296000      59200.0

SQL> SELECT city,
        COUNT(*) AS employee_count
FROM employee
GROUP BY city
HAVING COUNT(*) > 1
ORDER BY city;
city            employee_count
-----
Bangalore      2
Chennai        2
Delhi          2
Hyderabad      2
Mumbai         2

```

8. Results

Sixteen employees load successfully. Every question returns a non-empty result on this seed. The figures below are the row counts and values from the SQLite run in section 7.

Q. No.	Rows	What the result shows
1	—	Table employee created with 6 attributes
2	16	16 rows inserted across CSE, Mechanical, ECE and Civil
3	1	COUNT(*) = 16 employees
4	1	COUNT(DISTINCT department) = 4
5	1	SUM(salary) = 898000
6	1	AVG(salary) = 56125.0
7	1	MAX = 92000 and MIN = 28000
8	4	COUNT, SUM, AVG, MAX and MIN for each of the 4 departments
9	3	CSE (59200.0), ECE (58750.0), Mechanical (58000.0); Civil excluded
10	3	CSE (5), ECE (4), Mechanical (4); Civil has only 3
11	1	CSE only: 5 employees, total 296000, average 59200.0
12	5	Bangalore, Chennai, Delhi, Hyderabad and Mumbai, 2 each

Department totals from question 8 add up to the overall total: 296000 + 135000 + 235000 + 232000 = 898000, and 5 + 3 + 4 + 4 = 16. The CSE row of that result is the same triple question 11 prints on its own (5, 296000, 59200.0). Questions 9 and 10 both drop Civil, for different reasons: its average is 45000.0, and it has only three employees. Question 12 keeps the five cities that are shared and drops Ahmedabad, Pune, Jaipur, Kochi, Kolkata and Nagpur.

9. Conclusion

Aggregate functions collapse many rows into one value. COUNT answers how many, SUM and AVG answer how much, and MAX / MIN answer the extremes. GROUP BY repeats that collapse once per group. WHERE chooses the rows that enter the calculation; HAVING chooses which groups survive it. These are the tools for summary questions. Questions that compare a row with a computed value — more than the average, the highest in each department — are nested queries, and they are Experiment 6.